



Bubble Sort

Principles of Comparative Sorting in Python

Series 10: Order & Arrangement

Authored by: CODEHIVE CONTENT LABS

01. Why 'Bubble'?

Bubble Sort is a comparison-based sorting algorithm. It derives its name from the way larger elements "bubble up" to their correct positions at the end of the list during each iteration.

The Core Mechanism

1. Compare adjacent elements.
2. If the left element is larger than the right, swap them.
3. Repeat the process for every element in the list.
4. After one full pass, the largest element is guaranteed to be at the final index.

Key Insight: With each completed pass, the algorithm needs to look at one fewer element than before, as the "tail" of the list becomes increasingly sorted.

02. Manual Execution

Let's trace a single pass of the array: [7, 12, 9, 11, 3]

Step 1: Compare 7 and 12. Position is correct. No swap.

Step 2: Compare 12 and 9. $12 > 9$, so **swap**.

Result: [7, 9, 12, 11, 3]

Step 3: Compare 12 and 11. $12 > 11$, so **swap**.

Result: [7, 9, 11, 12, 3]

Step 4: Compare 12 and 3. $12 > 3$, so **swap**.

Result: [7, 9, 11, 3, 12]

Result of Pass 1:

The number **12** (the largest) has bubbled to the end. The next pass will focus only on the first four elements.

03. Standard Implementation

To implement this in Python, we use nested loops. The outer loop controls the number of passes, while the inner loop performs the comparisons and swaps.

```
mylist = [64, 34, 25, 12, 22, 11, 90, 5]
n = len(mylist)

for i in range(n-1):
    # n-i-1 because the last i elements are already sorted
    for j in range(n-i-1):
        if mylist[j] > mylist[j+1]:
            # Pythonic Swap
            mylist[j], mylist[j+1] = mylist[j+1], mylist[j]

print(mylist)
```

04. The "Early Exit" Optimization

The standard Bubble Sort is inefficient because it continues to run even if the list becomes sorted halfway through the process. We can improve this using a Boolean flag.

```
for i in range(n-1):
    swapped = False
    for j in range(n-i-1):
        if mylist[j] > mylist[j+1]:
            mylist[j], mylist[j+1] = mylist[j+1], mylist[j]
            swapped = True

    # If no two elements were swapped, the list is sorted!
    if not swapped:
        break
```

Efficiency Boost: This allows the algorithm to finish in **$O(n)$** time if the array is already sorted.

05. Big O Analysis

Bubble Sort is generally considered an inefficient algorithm for large datasets due to its quadratic time complexity.

$O(n^2)$

Performance Metrics:

- **Worst Case:** $O(n^2)$ - Array is sorted in reverse.
- **Average Case:** $O(n^2)$
- **Best Case (Optimized):** $O(n)$ - Array is already sorted.
- **Space Complexity:** $O(1)$ - Only requires a tiny amount of extra memory for the swap variable.

06. Conclusion & Use Cases

While Bubble Sort is rarely used in production software, it serves as an essential teaching tool for understanding the basics of sorting logic and complexity analysis.

When to use Bubble Sort?

- For **educational purposes** to learn swap logic.
- When the dataset is **extremely small**.
- When you need an algorithm that is **stable** (maintains relative order of equal elements) and uses minimal memory.

CODEHIVE ACADEMY

© 2026. Data Engineering series. Document ID: SRT-BBL-010.